

seL4 Summit 2022

As part of the bootcamp on day 4 at the seL4 summit 2022, two workshops will be contributed by HENSOLDT Cyber (HC):

- CAMkES Session: 10:15 - 10:45
- TRENTOS Session: 10:45 - 12:00

The overall goal is to provide the audience with a contiguous task, spanning from the standard CAMkES environment to its integration within TRENTOS. Therefore, at first, a small application is to be developed based on the mainline seL4 and CAMkES setup. This application is then transitioned into the TRENTOS environment, which allows for reusing the existing C code and CAMkES descriptions and extending them with additional OS features provided by TRENTOS.

Development Setup

As up to 10 participants are expected to take part in person, a respective development environment has to be prepared and provided by HC accordingly.

Hardware Setup

The following hardware setup has to be provided by HC:

- 12 HP laptops from HC
- 12 external mouse peripherals
- WiFi internet access

Software Setup

The following software setup has to be provided by HC:

- Ubuntu Linux (20.04 or 22.04) pre-installed
- seL4 and CAMkES docker environment installed: <https://docs.sel4.systems/projects/dockerfiles/>
- TRENTOS SDK 1.3 pre-installed
 - build docker container
 - test docker container
- potentially required host dependencies pre-installed

CAMkES Session (30 minutes)

The CAMkES session will be split into two parts: a short theory section that explains the required CAMkES architecture and corresponding building blocks and the actual practical session, where participants are required to build and run a small CAMkES application within a QEMU ARM environment.

Theoretical Part

As part of the theoretical part, the following aspects shall be highlighted and explained:

- CAMkES idea, architecture and origin
- abstraction from the seL4 kernel (kernel objects, IPC, signal, shared memory)
- basic building blocks
 - components
 - connections/interfaces
 - compositions/assembly
- keyword concept

Practical Part

As part of the practical part, a small CAMkES application shall be designed and implemented, which makes use of

- a custom interface, defining two specific functions (read, write)
- two components (client, server) that utilize the interface and communicate via IPC (RPC) and shared memory (dataport)
 - the server component has to implement the logic of the functions defined by the interface
 - the client component has to use the functionality via RPC call
 - actual data exchange is provided via a dataport

The components thereby have to fulfill different tasks, with

- the client
 - using the defined interface by calling
 - write() to transfer a String to the server
 - read() to receive back a "modified" String from the server
 - visualizing the received data on the console
- the server, in case of

- a call to the write() function
 - receiving a String from the client component
 - modifying the String (via String reverse)
 - storing the String in an array
- a call to the read() function
 - reading back the modified String from the array
 - forwarding the modified String back to the client component

By using this design, the participants shall

- build a small example that
 - defines an interface (using IDL)
 - declares two components (using ADL)
 - provides a composition with
 - two component instances
 - respective connections between the instances (RPC and dataport)
 - provides an assembly
 - provides additional C code logic
- run build process
 - via official docker container
 - for the build target: QEMU ARM "sabre"
- run the example with the help of
 - the seL4 and CamkES docker container (following the official instructions and recommended development path)
 - using the QEMU ARM "sabre" platform

TRENTOS Session (75 minutes)

The TRENTOS session will be split into two parts: a short theory section that explains the required TRENTOS architecture and corresponding building blocks and the actual practical session, where participants are required to transfer the previous CamkES setup into the TRENTOS environment and build and run a small TRENTOS application within the already used QEMU ARM setup.

Theoretical Part

As part of the theoretical part, the following aspects shall be highlighted and explained:

- the TRENTOS idea (abstract from seL4 and CamkES; add typical OS support with focus on secure embedded systems)
- its architecture (using graphical figures) -> focus on modularity due to components; easy replacement and porting, as only specific parts of a system have to be modified in case
- the SDK structure
 - build and test
 - source code
 - seL4 and CamkES dependencies
 - TRENTOS API and libraries
 - TRENTOS standard components
 - device drivers
 - additional software ...
 - TRENTOS demo systems
 - documentation
 - helper stuff
- the steps required to build an own application

Practical Part

As part of the practical part, a small TRENTOS application shall be designed and implemented, which transfers the previous CamkES application into the TRENTOS environment in order to

- show the possibility for an easy reuse of a CamkES setup
- build and run the example with the help of
 - the TRENTOS build container, for the build target: QEMU ARM "sabre"
 - the TRENTOS test container (again using the QEMU ARM "sabre" platform)

Afterwards, the existing application shall be extended via TRENTOS features by

- keeping the client component as is
 - using the defined interface by calling
 - write() to transfer a String to the server
 - read() to receive back a "modified" String from the server
 - visualizing the received data on the console
- transforming the original server component into a man-in-the-middle component, which in case of
 - a call to the write() function
 - receives a String from the client component
 - modifies the String (via String reverse)
 - utilizes the TRENTOS Crypto API in order to encrypt the modified String
 - stores the encrypted String, with the help of the TRENTOS FileSystem API, in a storage backend by using the TRENTOS RamDisk component and the if_OS_Storage interface
 - a call to the read() function

- reads back the encrypted String from the storage backend with the help of the TRENTOS FileSystem API
- utilizes the TRENTOS Crypto API in order decrypt the encrypted String
- forwards the modified and decrypted String back to the client component

The TRENTOS application thereby makes use of

- the TRENTOS API
 - TRENTOS Crypto API
 - TRENTOS FileSystem API
- TRENTOS standard components
 - the RamDisk component
 - the EntropySource component (as required by the TRENTOS Crypto API)
- TRENTOS interfaces
 - the TRENTOS **if_OS_Storage** interface
 - the TRENTOS **if_OS_Entropy** interface

Finally, the participants shall build and run the example with the help of

- the TRENTOS build container, for the build target: QEMU ARM "sabre"
- the TRENTOS test container (again using the QEMU ARM "sabre" platform)